Heart Disease Prediction

MLOps project report | Cleveland dataset

github.com/arunbaby84/heart_disease_prediction

1. Setup and installation

Requirements: Python 3.11 or newer. Docker Desktop is optional for local services; a Kubernetes cluster is required to apply the deployment manifest.

Windows PowerShell

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

Linux / macOS

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Download, preprocess, train, and serve

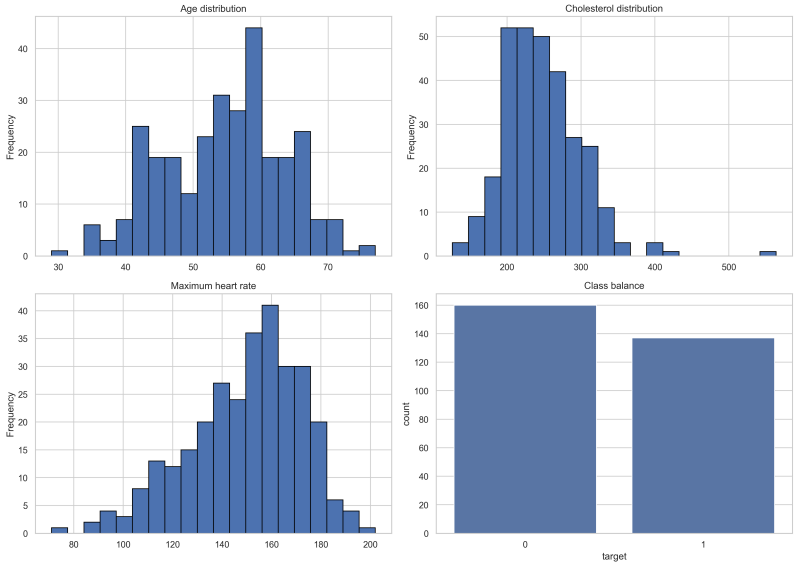
For Windows, run each command through .venv\Scripts\python.exe instead of python.

```
python scripts/download_data.py
python scripts/preprocess_data.py
python scripts/train_model.py
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Run docker compose up --build for the API, MLflow, Prometheus, and Grafana services. Local service ports are 8000, 5000, 9090, and 3000 respectively.

2. Dataset, EDA, and modeling

The downloader reads the Cleveland heart disease CSV from the UCI archive, converts '?' to missing values, coerces columns to numeric, drops incomplete rows, and maps target values above zero to the positive class. EDA covers distributions, class balance, correlations, pairwise feature relationships, and IQR outliers.

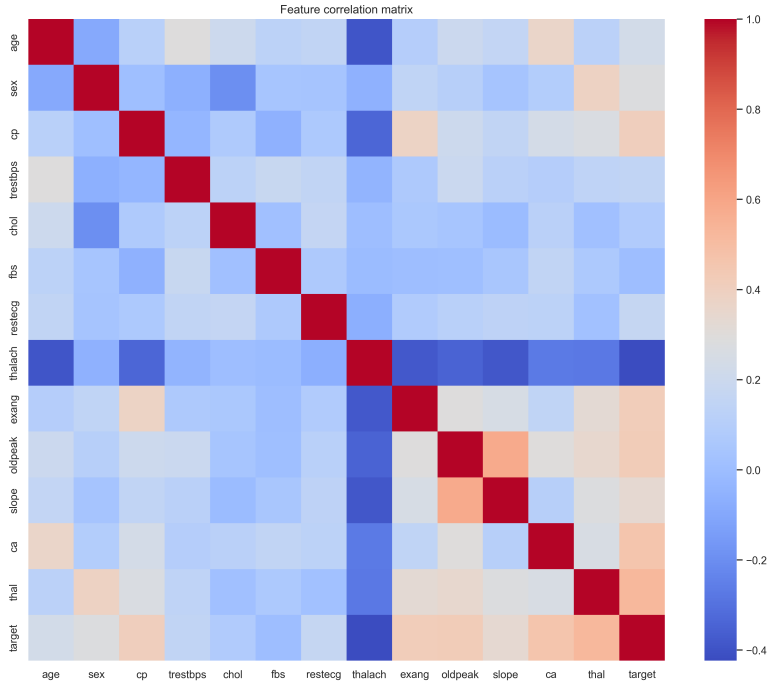


EDA overview: age, cholesterol, maximum heart rate, and class balance.

Candidate pipelines combine median imputation and standard scaling with Logistic Regression or a class-weighted Random Forest (300 trees, minimum leaf size 2). A fixed-seed stratified 80/20 split supplies holdout metrics; five-fold stratified cross-validation on training data estimates ROC-AUC stability.

Model	Accuracy	Precision	Recall	Holdout AUC	CV AUC mean (std)
Logistic Regression	0.833	0.846	0.786	0.950	0.869 (0.068)
Random Forest	0.867	0.885	0.821	0.942	0.894 (0.059)

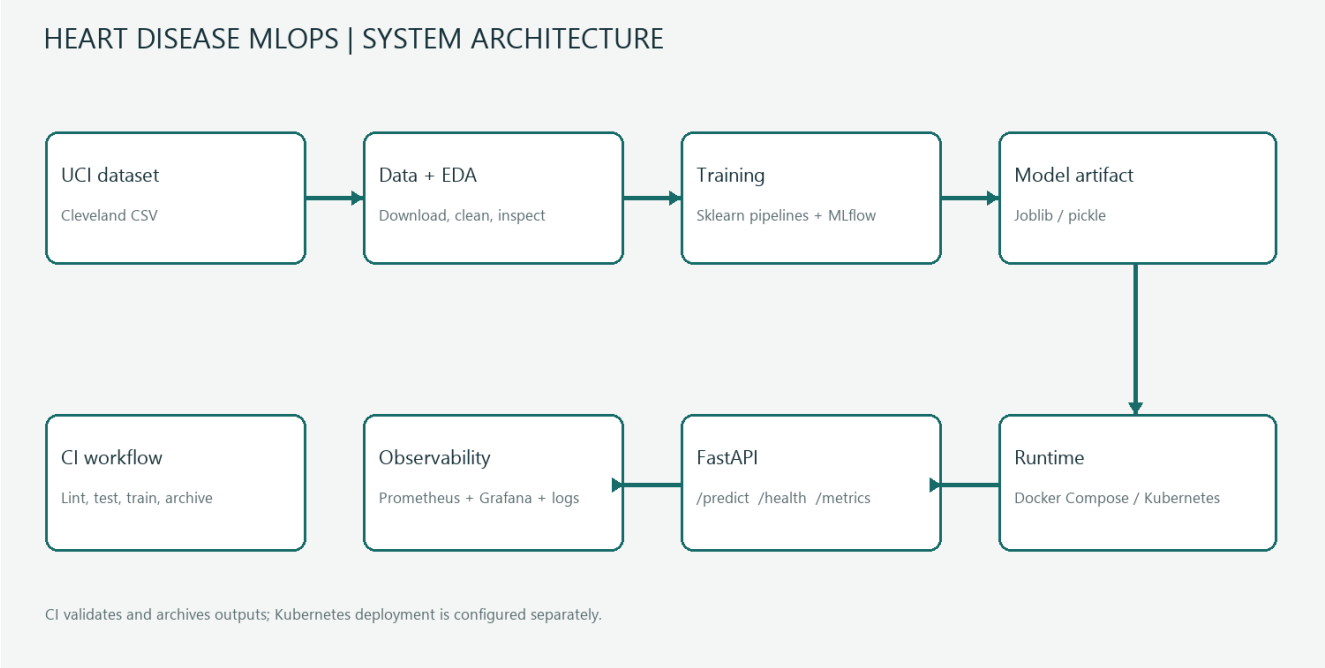
Current selection uses holdout ROC-AUC, so Logistic Regression is selected by the recorded metrics. Random Forest leads on accuracy, precision, recall, and mean cross-validation ROC-AUC. The small dataset and narrow holdout difference make these educational results, not clinical evidence.



Feature correlation matrix.

3. Experiment tracking and architecture

Each training execution logs two candidate runs to the heart_disease_prediction MLflow experiment. Runs include estimator parameters, holdout metrics, cross-validation ROC-AUC mean and standard deviation, and a serialized scikit-learn pipeline with inferred signature. The local store is mlruns/. Model files are artifacts/best_model.joblib and artifacts/best_model.pkl; metrics are summarized in artifacts/model_report.json. Browse runs with python -m mlflow ui --host 127.0.0.1 --port 5000.



Architecture and workflow boundaries. CI validates and archives outputs; cluster deployment is configured separately.

The FastAPI service shares the saved preprocessing-and-estimator pipeline. /predict returns a class and positive-class probability; /health supports probes; /metrics exposes Prometheus instrumentation. Middleware logs request ID, method, path, status, and duration.

4. CI/CD and deployment

GitHub Actions runs for pushes and pull requests to main or master. It sets up Python 3.11, installs requirements, runs Ruff, downloads data, runs pytest, and trains the models. Workflow reports, model outputs, MLflow runs, and processed data are uploaded as artifacts. The workflow does not publish a container image or automatically deploy to Kubernetes.

GitHub Actions CI workflow | source evidence

Source: .github/workflows/ci.yml

```
name: Heart Disease MLOps CI

on:
  push:
    branches: [ main, master ]
  pull_request:
    branches: [ main, master ]

...
  push:
  pull_request:
      - name: Set up Python 3.11
      - name: Install dependencies
      - name: Lint code
      - name: Download dataset
      - name: Run unit tests
      - name: Train model
      - name: Upload workflow reports and model artifacts
```

Configuration-derived source evidence, not a screenshot of a live GitHub Actions run.

Kubernetes configuration requests two API replicas, uses /health for readiness and liveness probes, and exposes port 8000 through a LoadBalancer Service. After making the referenced image available to a configured cluster, apply it with kubectl apply -f k8s/deployment.yaml.

Kubernetes deployment manifest | source evidence

Source: k8s/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: heart-disease-api
spec:
  replicas: 2
  selector:
    matchLabels:
...
kind: Deployment
  replicas: 2
          image: heart-disease-prediction:latest
          readinessProbe:
          livenessProbe:
kind: Service
  type: LoadBalancer
```

Configuration-derived source evidence, not a screenshot of a running cluster.

5. Reproducibility and limitations

Pinned dependencies and fixed random seeds make the workflow repeatable. The saved full pipeline keeps imputation and scaling consistent between training and inference. Run tests with python -m pytest -q. Docker Compose includes MLflow, the API, Prometheus, and Grafana with provisioned dashboards.

Evidence status

The repository includes EDA figures but no captured live GitHub Actions run or deployed-cluster dashboard. The two workflow images in this report are labeled renderings of checked-in configuration, not fabricated operational screenshots. Capture real run and cluster screens from the target GitHub repository and environment if the assignment requires proof of a successful live run or deployment.

This dataset and model are for educational demonstration only. Any real healthcare use would require external validation, calibration, subgroup analysis, privacy and security review, and clinical governance.

Repository: https://github.com/arunbaby84/heart_disease_prediction